Activation Planner — Kickoff Memo

Read this first, before the accompanying Activation_Planner_Reference.pdf. This memo tells you how to use that document and what state this project is actually in.

What This Project Is

Activation Planner is a new, standalone application (C#/.NET 10, cross-platform: Windows, macOS, Linux) for planning ham radio operating sessions — POTA activations, Field Day, SOTA, EMCOMM deployments, or any general operating session. It helps the operator decide which bands are likely to work (using real propagation science, not guesswork), which antenna from their own inventory suits the conditions, and what gear to pack, with an editable checklist.

It began as a brainstormed idea during work on a separate, existing project called IcomRigControl (a mature, ten-phase ham radio transceiver control application, also C#/.NET 10, also cross-platform). Activation Planner is deliberately being built as its own separate application, not a feature bolted onto IcomRigControl — see the reference document, Section 2, for the reasoning.

How to Use the Reference Document

Activation_Planner_Reference.pdf is a thorough record of everything discussed and researched so far: 11 sections covering the standalone-vs-integrated decision, the tech stack, propagation prediction (VOACAP) research including a real licensing question that was resolved, GPS input, POTA website integration, the gear checklist design, hardware Jim owns (Mobilinkd TNC4, Digirig Mobile, SignaLink USB), the EMCOMM use case, and a full open-items list.

It uses a color-coded tagging system throughout — read this carefully, since it distinguishes what's actually been decided from what's merely been suggested:

- **Green** = Jim's own idea or request.
- **Blue** = Claude's suggestion or research finding — NOT yet a decision.
- **Purple** = Something the two of them decided together. Treat these as settled.
- **Orange** = Genuinely open, not yet decided. Do NOT treat these as settled — ask Jim before proceeding as if a decision has been made.

Section 11 of the reference document ("Summary: Open Items and Next Steps") is the single most important section to read before writing any code — it lists every unresolved question that should be raised with Jim rather than assumed.

What's Genuinely Decided (Purple items, safe to treat as settled)

- Standalone application, not a feature inside IcomRigControl.
- C# and .NET 10, cross-platform (Windows/macOS/Linux).
- Named 'Activation Planner' — deliberately renamed from an earlier 'POTA Activation Planner' working name, because the feature set (propagation, antenna selection, gear checklists) is useful for any operating session, not just POTA.

- Jim will license the project under AGPLv3 or GPLv3 (not yet decided which one specifically). Propagation prediction will use standalone VOACAP, run locally and invoked as a separate process (never linked or embedded), which keeps this licensing choice unaffected — see reference Section 4.3 for the full reasoning.

What's Genuinely NOT Decided Yet (do not assume)

- UI framework (Avalonia, matching IcomRigControl, or something else).
- The precise 'planning-only, no logging duplication' boundary hasn't been explicitly confirmed by Jim, only suggested by Claude.
- Whether/how to integrate the Mobilinkd TNC4, Digirig Mobile, and SignaLink USB hardware Jim owns.
- Whether antenna modeling starts with the simpler community-library approach or goes straight for NEC2++ custom modeling (both are real, researched, viable paths — see reference Section 4.5).
- The relationship between this tool and Jim's existing EMMCOM Field Comms Server work in IcomRigControl.
- The actual POTA API details (endpoints, auth, rate limits) haven't been researched yet at all — only the general concept has been discussed.

Working Style Established With Jim (from IcomRigControl sessions)

Jim and Claude have an established, effective working pattern from building IcomRigControl over many sessions, worth continuing here:

- Test-driven development: write tests first, confirm they fail for the right reason, then implement, then confirm they pass. This discipline was used consistently and successfully throughout IcomRigControl (hundreds of passing tests across ten feature phases).
- Research before building, especially for external protocols/formats/APIs — the VOACAP research in the reference document (input file format, licensing, antenna modeling options) is a good example of the depth expected before writing code against something unfamiliar.
- Verify claims with real evidence rather than assumption — e.g. actually checking VOACAP's stated automation policy before assuming it was fine to script.
- When something is genuinely uncertain or a real decision point, say so plainly and ask, rather than picking an answer and presenting it as settled.
- Jim runs all commands himself (build, test, commit) and reports results back verbatim; Claude does not have direct access to Jim's development machines in these sessions.

Suggested First Steps in the New Project

- Read the full reference document before writing any code.
- Raise the open items from Section 11 with Jim early, rather than guessing at defaults, especially the UI framework choice and the planning-only design boundary, since those affect the whole architecture.

- Once those are settled, a sensible first real milestone (not yet discussed with Jim, offered here only as a starting suggestion) would be the VOACAP shell-out piece: writing a correctly-formatted input deck, running standalone VOACAP as an external process, and parsing its output — since it's the most research-heavy, foundational piece the rest of the app depends on.